

WannaCry Ransomware Worm — Cyber Range Attack Simulation Design Document

Scenario ID: W1 — WannaCry-Style SMB Worm **Classification:** Self-propagating ransomware (worm + crypto-locker) **Document type:** Simulation design specification (consumed by a software development team) **Reference baseline:** MITRE ATT&CK techniques as listed in the attack flow **Simulation intent:** Visually and behaviorally recreate the *unfolding* of a WannaCry-class incident from both attacker and defender vantage points. This is a software simulation. It contains no exploit code, no malware, and no weaponizable detail — only the observable behavior, telemetry, and state changes a development team needs to model.

Design north star: Every stage must be expressible as a *state transition* with associated *attacker output*, *victim experience*, *SOC-visible telemetry*, and *defender decision options*. If a stage cannot be reduced to those four lenses, it is under-specified.

3. Attacker Journey (action sequence) – Red Team Journey

The "attacker" in this simulation is a **deterministic worm automaton**, not a live human operator. Every action is scripted and repeats per host. The sequence below is the canonical loop a development team should model as the attacker engine.

#	Attacker Action	Trigger	Target	Result
1	Enumerate reachable hosts on local subnet(s)	New infection reaches "active" state	Subnet address range	List of live hosts
2	Probe candidates for vulnerable SMBv1 service on TCP/445	Per discovered host	Each live host	Vulnerable / not-vulnerable verdict
3	Attempt exploitation of vulnerable host	Per vulnerable host	Single target	Remote code execution foothold (simulated)
4	Transfer + drop payload to target	Successful exploit	Target host	Payload present on disk
5	Register persistence (service/auto-run)	Payload dropped	Target host	Payload survives reboot
6	Perform outbound C2 / kill-switch check	Payload executes	External domain	Continue / abort decision
7	Spawn worm thread on new host	C2 check passes	New host	New host enters propagation loop (return to step 1)
8	Disable recovery (shadow copies, backup catalog, recovery env)	Pre-encryption	Local host	Recovery options removed

#	Attacker Action	Trigger	Target	Result
9	Enumerate + encrypt target file types	Recovery disabled	Local filesystem	Files encrypted + renamed
10	Drop ransom note + display lock screen	Encryption complete	Local host	Extortion demand presented

The crucial modeling property: **steps 1–7 (spread) and steps 8–10 (destruction) run in parallel across the host population.** A host can be encrypting itself while simultaneously infecting neighbors. The simulation must not serialize these globally.

4. Victim Journey (per-host experience)

What a single infected host experiences, in order, from the user's and the machine's perspective:

1. **Silent compromise.** No visible symptom. CPU and disk normal. The user keeps working. (This is the most important UX beat to simulate — the victim has *no idea*.)
2. **Background activity begins.** Slight, often-unnoticed network and disk activity as the worm scans and the dropper installs.
3. **Brief pause / micro-freeze.** Some hosts show momentary unresponsiveness as recovery is disabled and encryption spins up.
4. **Files start changing.** Document icons go blank/generic; filenames gain a new extension; files become un-openable. Applications throw "file corrupted" errors.
5. **Ransom note appears.** Text/HTML note files materialize in every affected folder.
6. **Full-screen lock.** A ransom window seizes the display, showing a countdown timer and payment demand. The host is now effectively unusable for normal work.
7. **Reboot does not help.** Persistence ensures the payload returns; encrypted files stay encrypted; recovery options are gone.

State the host moves through: Healthy → Vulnerable → Exploited → Infected → (Propagating || Encrypting) → Impacted.

5. SOC Analyst Journey (what's visible, when)

The SOC view is intentionally designed as a **detection funnel** — signals grow louder as damage grows. This teaches that the *cheapest* detection is the *earliest* one.

Stage	What lands in the SIEM/console	Signal strength	Typical analyst reaction
Network Discovery	Spike in TCP/445 connection attempts from one source; horizontal scan pattern	Low / easy to miss	Often deprioritized as noise
SMB Target ID	Repeated SMB session setups to many hosts; unusual SMBv1 negotiation	Low–Medium	Possible "scanning" alert

Stage	What lands in the SIEM/console	Signal strength	Typical analyst reaction
Exploit	IDS/IPS exploit-pattern signature on SMB traffic	Medium-High	Should trigger investigation
Payload Deployment	Suspicious file write to system directory; new binary on host	Medium	EDR alert if tuned
Persistence	New service / autorun creation event	Medium	EDR / Sysmon alert
C2 Callback	Outbound DNS/HTTP to suspicious external domain	Medium	Proxy/DNS alert
Lateral Movement	Same scan/exploit pattern now from <i>multiple</i> internal sources	High	Clear "this is spreading" signal
Disable Recovery	Shadow-copy deletion + backup-catalog commands	High / unambiguous	Strong ransomware indicator
Encrypt Files	Mass file-rename/modify burst; new extension; ransom-note file creation	Very High	Confirmed ransomware
Business Impact	Host availability alarms, service-down monitors, help-desk surge	Critical	Major incident declared

Key SOC training insight: the alerts that *confirm* the attack (shadow-copy deletion, mass encryption) arrive *after* prevention is impossible. The alerts that could *stop* it (port-445 scan, exploit signature) arrive early but quiet. The simulation should explicitly score the analyst on how early in this funnel they act.

6. Blue Team Journey (defender actions available)

For each stage, the simulation exposes a set of defender "moves." Each move has a cost (time, business disruption) and an effect on the worm population. This is the interactive heart of the cyber range.

Stage reached	Detect	Investigate	Contain	Eradicate
Discovery / Scan	Tune scan-detection rule	Pivot on source host	Isolate source host (NAC/EDR)	Patch + disable SMBv1 fleet-wide
Exploit	Confirm IDS signature	Triage exploited host	Block TCP/445 at segment boundary	Push MS17-010-class patch
Payload / Persistence	EDR detonation alert	Inspect dropped artifact	Quarantine host	Kill service + remove persistence
C2	DNS/proxy alert	Identify callback domain	Sinkhole / block domain (the historical "kill switch")	—

Stage reached	Detect	Investigate	Contain	Eradicate
Lateral Movement	Correlate multi-source pattern	Map infected set	Network segmentation — sever VLAN links	Mass isolate infected hosts
Disable Recovery	Shadow-delete alert	Confirm pre-encryption stage	Emergency host isolation	Preserve backups offline
Encryption	Mass-rename alert	Scope encrypted hosts	Isolate to stop spread	Begin restore from clean backup
Impact	Availability alarms	Damage assessment	Full segment quarantine	Rebuild + restore

Highest-leverage defender moves to model prominently:

- **Disable/patch SMBv1 across the fleet** (kills the propagation vector entirely).
- **Network segmentation** (the single most effective containment once spread starts — converts exponential growth into a bounded blast radius).
- **Kill-switch domain sinkhole** (a stage-6 move that, in this worm's logic, halts further encryption — a deliberately included "lucky break" mechanic).
- **Offline/immutable backups** (the only move that meaningfully reduces *recovery* cost after encryption).

7. Detailed Stage Breakdown

Each stage below is specified across the ten lenses the development team needs. Telemetry is described at the *event-class* level (what a SIEM/EDR would record), not as exploit detail.

Stage 1 — Network Discovery (T1046)

- **Objective:** Find live hosts reachable from the infected machine.
- **Inputs:** Local subnet/CIDR ranges of the infected host.
- **Actions:** Sweep address range with connection probes (notably to TCP/445).
- **Outputs:** List of responsive hosts.
- **State changes:** Host: *Infected* → *Propagating*. Neighbors: unchanged (still *Healthy/Vulnerable*).
- **Telemetry:** High count of short-lived outbound TCP/445 connections from a single source; horizontal scan pattern in flow logs.
- **Detection opportunities:** Internal port-scan / fan-out detection; one-source-to-many-destinations heuristic.
- **Visual representation:** Pulsing "ping" rays radiating from the infected node to neighbors across the topology map.
- **Network changes:** Burst of connection setups; minor bandwidth uptick.
- **Business impact:** None yet (this is why it's dangerous — invisible to the business).

Stage 2 — SMB Target Identification

- **Objective:** Filter live hosts down to those running the vulnerable SMBv1 service.

- **Inputs:** List of live hosts from Stage 1.
- **Actions:** Negotiate SMB sessions; identify hosts exposing the legacy/vulnerable service version.
- **Outputs:** Subset of "vulnerable" hosts.
- **State changes:** Identified neighbors: tag as `Vulnerable` (target candidates) in the sim model.
- **Telemetry:** Repeated SMB session setups; SMBv1 dialect negotiation events; unusual SMB volume from one source.
- **Detection opportunities:** Alert on SMBv1 negotiation (legacy protocol = red flag in modern nets); many-target SMB enumeration.
- **Visual representation:** Neighbor nodes flip from gray (unknown) to amber (vulnerable) or stay green (patched).
- **Network changes:** Sustained SMB handshake traffic.
- **Business impact:** None yet.

Stage 3 — Exploit Vulnerable Host (T1210)

- **Objective:** Gain code execution on a vulnerable target.
- **Inputs:** A target tagged `Vulnerable`.
- **Actions:** Send the exploit pattern that abuses the SMBv1 flaw to obtain remote execution. (*Modeled as a success/fail probability + IDS-signature emission; no exploit mechanics simulated.*)
- **Outputs:** Foothold on target; ability to deliver payload.
- **State changes:** Target: `Vulnerable` → `Exploited`.
- **Telemetry:** IDS/IPS exploit-pattern signature on SMB traffic; anomalous SMB packet sequence.
- **Detection opportunities:** Signature-based IDS alert (highest-confidence early signal); IPS can *block* here if inline.
- **Visual representation:** A "strike" animation along the edge; target node flashes red briefly, then turns orange (Exploited).
- **Network changes:** Distinct exploit-pattern flow on port 445.
- **Business impact:** None to user yet; the machine is compromised but symptom-free.

Stage 4 — Payload Deployment (T1059/003)

- **Objective:** Place the ransomware/worm payload on the exploited host.
- **Inputs:** Exploited host foothold.
- **Actions:** Transfer and write the payload to a system directory; launch it via command interpreter.
- **Outputs:** Payload present and running.
- **State changes:** Host: `Exploited` → `Infected`.
- **Telemetry:** File-creation events in system dirs; new-process / process-creation events; command-interpreter spawn from an unexpected parent.
- **Detection opportunities:** EDR file-write + process-lineage alerts; "system service spawned cmd/script" anomaly.
- **Visual representation:** Node turns solid red; a small package icon "drops" onto the node.
- **Network changes:** Short inbound file transfer.
- **Business impact:** None visible yet.

Stage 5 — Establish Persistence (T1059/003)

- **Objective:** Survive reboot and re-execute.
- **Inputs:** Running payload.
- **Actions:** Register a service / auto-run entry pointing at the payload.
- **Outputs:** Persistent execution mechanism.
- **State changes:** Host gains `persistent=true` attribute (stays `Infected`).
- **Telemetry:** Service-creation event; autorun/registry-run modification event.
- **Detection opportunities:** New-service and autorun-change alerts (classic persistence detections).
- **Visual representation:** A small "anchor/pin" badge appears on the node.
- **Network changes:** None.
- **Business impact:** Raises *recovery cost* — reimaging now required, not just process kill.

Stage 6 — Command & Control Callback (T1071/001)

- **Objective:** Perform the worm's external check (the "kill-switch" lookup).
- **Inputs:** Running payload.
- **Actions:** Single outbound DNS/HTTP request to a hardcoded domain. If the domain resolves/responds, the worm's logic *halts further malicious action*; if not, it proceeds. (*This is the famous accidental off-switch — include it as a scenario mechanic.*)
- **Outputs:** Continue-or-abort decision.
- **State changes:** Branch: domain reachable → host stalls (`Infected`, dormant); unreachable → proceeds to encryption.
- **Telemetry:** Outbound DNS query / HTTP GET to an unusual, newly-seen domain.
- **Detection opportunities:** DNS/proxy anomaly; newly-registered-domain alert.
Defender move: sinkholing the domain triggers the kill switch fleet-wide.
- **Visual representation:** A dashed line shoots from node to a "C2 / kill-switch" icon at the map edge; if defender sinkholes it, affected nodes turn from red to gray-frozen ("neutralized").
- **Network changes:** Single distinctive outbound connection per host.
- **Business impact:** None directly — but this is a *decisive defender opportunity*.

Stage 7 — Lateral Movement (T1021/002)

- **Objective:** Reach hosts beyond the original subnet.
- **Inputs:** Infected host + its propagation thread.
- **Actions:** Repeat Stages 1–3 against adjacent subnets / via SMB shares.
- **Outputs:** New exploited hosts in new network segments.
- **State changes:** New hosts: `Healthy/Vulnerable` → `Exploited`. Worm population grows.
- **Telemetry:** Scan + exploit patterns now originating from **multiple internal sources** simultaneously.
- **Detection opportunities:** Multi-source correlation ("the same attack is coming from 5 internal hosts" = spreading worm). This is the unambiguous "we are under active worm propagation" signal.
- **Visual representation:** Multiple nodes simultaneously emit scan rays; the "red zone" visibly expands across VLAN boundaries.
- **Network changes:** Cross-segment SMB traffic; east-west traffic spike.

- **Business impact:** Beginning of widespread risk; still pre-encryption on newly hit hosts.

Stage 8 — Repeat Propagation

- **Objective:** Sustain exponential growth.
- **Inputs:** Every infected host's propagation loop.
- **Actions:** Each infected host independently runs Stages 1–7.
- **Outputs:** Geometric increase in infected count until vulnerable hosts are exhausted or containment intervenes.
- **State changes:** Population-level: infected count grows ~exponentially.
- **Telemetry:** Network-wide port-445 storm; SMB traffic saturation.
- **Detection opportunities:** Aggregate volume anomaly; segment-level traffic alarms.
- **Visual representation:** The topology map "lights up" in waves; an infection-count graph climbs steeply.
- **Network changes:** Possible congestion / degraded performance from scan volume.
- **Business impact:** Network performance degradation begins; helpdesk tickets about slowness.

Stage 9 — Disable Recovery (T1490)

- **Objective:** Prevent the victim from restoring files locally.
- **Inputs:** Infected host (pre-encryption).
- **Actions:** Delete volume shadow copies; clear backup catalog; disable the recovery environment via administrative commands.
- **Outputs:** Local recovery options removed.
- **State changes:** Host gains `recovery_disabled=true` (still Infected, now committed to encryption).
- **Telemetry: High-fidelity** command-line events: shadow-copy deletion, backup-catalog deletion, recovery-environment disable. Sysmon/EDR process-creation with these command lines.
- **Detection opportunities:** One of the strongest ransomware tells — alert on shadow-copy/backup-deletion commands. If caught *here*, isolation can still save files not yet encrypted.
- **Visual representation:** A "shield/backup" icon on the node shatters or greys out.
- **Network changes:** None.
- **Business impact:** Recovery cost spikes — restoration now depends entirely on *off-host* backups.

Stage 10 — Encrypt Files (T1486)

- **Objective:** Render business data inaccessible to force ransom payment.
- **Inputs:** Host with recovery disabled.
- **Actions:** Enumerate target file types (documents, spreadsheets, images, archives, databases); encrypt each with a hybrid symmetric+asymmetric scheme; rename with a distinctive extension; drop ransom notes; display lock screen. (*Modeled as file-state changes and rename events — no cryptographic routine is implemented in the sim.*)
- **Outputs:** Encrypted, renamed files; ransom notes; lock screen.
- **State changes:** Host: Infected → Encrypting → Impacted. Files: accessible → encrypted.

- **Telemetry:** Mass file-modify + rename burst; new file extension appearing en masse; ransom-note file-creation events; sudden high disk I/O.
- **Detection opportunities:** Mass-rename / entropy-spike detection; ransom-note filename signature; "many files changed by one process in seconds" heuristic. Detection here only limits *spread*, not local damage.
- **Visual representation:** Files in a host's mini-view flip to "locked" icons; node turns dark red/black with a lock glyph; ransom-note popup overlay.
- **Network changes:** Minimal (encryption is local).
- **Business impact: Direct and severe** — host's data unusable; user locked out.

Stage 11 — Business Impact

- **Objective:** (From the attacker's view) maximize disruption to force payment.
- **Inputs:** Population of impacted hosts.
- **Actions:** N/A (consequence stage).
- **Outputs:** Outages, downtime, financial loss.
- **State changes:** Hosts *Impacted*; dependent services *Down*.
- **Telemetry:** Availability monitors firing; service-health red; helpdesk ticket surge; SLA breach alarms.
- **Detection opportunities:** N/A for prevention — this is the cost-accounting stage.
- **Visual representation:** Business-services dashboard turns red; financial-loss counter accelerates; downtime clock runs.
- **Network changes:** Service endpoints unreachable.
- **Business impact:** Quantified in Section 9.

8. Propagation Model

Host states (for every node on the map)

Healthy · Vulnerable · Exploited · Infected · Propagating · Encrypting · Impacted · Dormant (kill-switch tripped) · Contained · Eradicated · Recovered

Infection logic (per target host)

```
if target.state == Vulnerable and source.state in {Infected, Propagating}:
    if exploit_success(p_exploit):          # probability gate; emits IDS
signature event
    target.state = Exploited
    deploy_payload(target)                  # -> Infected, emits file/process
telemetry
    establish_persistence(target)           # persistent = true
    if c2_check(target):                    # kill-switch domain reachable?
        target.state = Dormant             # neutralized branch
    else:
        disable_recovery(target)           # -> recovery_disabled = true
        start_encryption(target)           # -> Encrypting -> Impacted
        start_propagation(target)          # target begins infecting
neighbors
```

Spread logic (population level)

- Each `Infected` host spawns a propagation loop that scans its reachable subnets.
- Reachability is governed by the **network segmentation map** (see below). A host can only infect targets it can reach on TCP/445.
- Growth is **exponential** while vulnerable+reachable hosts remain: $infected(t+1) \approx infected(t) \times R$, where R (effective reproduction number) depends on (vulnerable density) $\times$ (reachability) $\times$ (p_exploit).
- Growth **slows and stops** when: vulnerable hosts are exhausted, OR segmentation severs reachability, OR the kill-switch domain is sinkholed (mass transition to `Dormant`), OR hosts are patched/isolated faster than they're infected.

Reachability / segmentation

- Model the network as VLANs/segments with explicit allow/deny edges for TCP/445.
- **Defender segmentation action** removes edges $\rightarrow$ instantly caps blast radius to already-infected segments.
- This is the single most important "knob" for teaching containment value.

Network visualization

- Force-directed topology graph, nodes colored by state.
- Animated scan rays (Discovery), strike animations (Exploit), expanding "red zone" (Propagation).
- Live **infection-count curve** and **R-value gauge** beside the map.
- Segment boundaries drawn as containers; severing one visibly "walls off" a region.

```
graph LR
  A[Healthy] --> B[Vulnerable]
  B -->|exploit success| C[Exploited]
  C -->|payload + persistence| D[Infected]
  D --> E{C2 / kill-switch reachable?}
  E -->|yes| K[Dormant / Neutralized]
  E -->|no| F[Propagating + Encrypting]
  F --> G[Impacted]
  D -.defender isolate.-> H[Contained]
  H -.reimage + patch.-> I[Eradicated]
  I -.restore backups.-> J[Recovered]
```

9. Business Impact Model

The simulation maintains a live cost ledger driven by host state. Suggested default parameters (all tunable by the developer/instructor):

Metric	Model	Default driver
Encrypted files	Σ files_per_host over Impacted hosts	e.g. 5,000–50,000 files/host
Impacted hosts	count of hosts in Encrypting/Impacted	grows with R
Service outages	dependency map: if $N\%$ of a service's hosts are Impacted, service = Degraded/Down	service-to-host mapping
Downtime	running clock per Down service until Recovered	minutes

Metric	Model	Default driver
Productivity loss	$\text{impacted_hosts} \times \text{loaded_hourly_cost} \times \text{downtime_hours}$	e.g. \$50–\$120/hr/user
Recovery cost	$\text{impacted_hosts} \times \text{reimage_cost} + \text{restore_effort}$	e.g. \$300–\$1,500/host
Service revenue loss	$\text{down_services} \times \text{revenue_per_min} \times \text{downtime_min}$	per critical service
Total financial impact	sum of the above + optional fixed IR/forensics cost	live counter

Dependency modeling: tag hosts with roles (workstation, file server, domain controller, DB, app server). Encrypting a *server* should cascade — e.g. a downed file server marks dependent workstations' shared-drive work as blocked, multiplying impact. This teaches *why* server-VLAN containment is the priority decision.

Outcome banding (for scoring/debrief):

- **Contained** — < ~10% of hosts impacted, critical services preserved.
- **Degraded** — partial spread, some critical service downtime, bounded recovery.
- **Catastrophic** — network-wide encryption, critical services down, multi-day recovery.

10. Simulation Timeline (minute-by-minute, illustrative)

A reference "no-intervention" timeline. The simulation should compress/expand this with a time-scale control, and **branch** the moment a defender acts.

T+	Event	State of network	SOC visibility
00:00	Patient-zero (FIN-WS-014) compromised	1 infected	— (silent)
00:30	Subnet scan begins	1 propagating	Faint port-445 spike (low-pri queue)
01:30	3 neighbors exploited	4 infected	IDS exploit signatures fire (uncorrelated)
02:30	Persistence + C2 checks on first wave	4 infected, persistent	DNS callbacks to unusual domain
03:30	Second wave: ~12 hosts	12 infected	Multi-source scan pattern emerges
04:30	Lateral movement crosses into HR VLAN	12 → spreading	"Spreading from multiple sources" signal
05:30	First hosts disable recovery	12 infected, recovery off	Shadow-copy-deletion alerts (HIGH)
06:30	First encryption begins; ransom notes drop	~20 infected, 4 encrypting	Mass-rename alerts; ransomware confirmed

T+	Event	State of network	SOC visibility
07:30	Exponential phase: ~40 hosts	40 infected, 20 impacted	Major incident declared
09:00	File server reached	server VLAN at risk	Availability alarms
12:00	Shared services down; helpdesk flooded	80+ impacted	Service-down monitors red
15:00	Saturation / vulnerable hosts exhausted	blast radius maxed	Full damage assessment

Branch markers (where defender action rewrites the timeline):

- Act by ~**02:00** (block 445 / isolate patient-zero) → likely **Contained**.
- Act by ~**05:00** (segmentation) → likely **Degraded**.
- Act after ~**07:00** → likely **Catastrophic**.
- Sinkhole C2 domain at any point → mass *Dormant*, halts *new* encryption (the kill-switch mechanic).

11. Decision Points

Each decision point is an interactive node with options, costs, and consequences the engine evaluates.

DP	When it surfaces	Detect	Investigate	Contain	Eradicate	Consequence of inaction
DP-1	Port-445 scan spike	Promote scan alert	Pivot on source	Isolate patient-zero	Disable SMBv1 + patch fleet	Worm reaches stage 3 on neighbors
DP-2	IDS exploit signature	Confirm signature	Triage exploited host	Block 445 at boundary	Patch vulnerable hosts	Payload deploys, persistence sets in
DP-3	C2 / kill-switch callback	DNS alert	Identify domain	Sinkhole the domain	—	Encryption proceeds unhalting
DP-4	Multi-source lateral movement	Correlate sources	Map infected set	Segment the network	Mass-isolate infected	Worm crosses into server VLAN
DP-5	Shadow-copy deletion	Shadow-delete alert	Confirm pre-encryption	Emergency isolate	Protect/verify offline backups	Files become unrecoverable locally
DP-6	Mass encryption / ransom notes	Mass-rename alert	Scope impact	Quarantine to stop spread	Begin restore from clean backups	Blast radius continues to grow

Scoring principle: earlier correct action = higher score, lower simulated loss. The debrief should overlay the learner's action timeline on the cost curve to show "the money you saved (or lost) by acting at minute X."

12. Simulation State Machine

Core required chain: Healthy → Vulnerable → Exploited → Infected → Propagating → Encrypting → Impacted, plus defender-driven and kill-switch branches.

```
stateDiagram-v2
    [*] --> Healthy
    Healthy --> Vulnerable: legacy SMBv1 exposed
    Vulnerable --> Exploited: exploit succeeds (p_exploit)
    Exploited --> Infected: payload deployed + persisted
    Infected --> Dormant: C2 kill-switch reachable / sinkholed
    Infected --> Propagating: C2 unreachable -> begins spreading
    Propagating --> Encrypting: recovery disabled (shadow copies gone)
    Encrypting --> Impacted: files encrypted + ransom note + lock screen

    Vulnerable --> Eradicated: patched / SMBv1 disabled (defender)
    Exploited --> Contained: isolated before payload (defender)
    Infected --> Contained: quarantined (defender)
    Propagating --> Contained: network segmented (defender)
    Encrypting --> Contained: isolated mid-encryption (limits spread)

    Contained --> Eradicated: kill process + remove persistence + reimage
    Dormant --> Eradicated: cleaned before reactivation
    Impacted --> Recovered: restore from clean/offline backup
    Eradicated --> Recovered: rebuild + restore
    Recovered --> [*]
```

State attributes the engine should carry per host: state, persistent (bool), recovery_disabled (bool), encrypted_file_count, role, segment_id, infected_at, impacted_at. **Transition guards** are the probabilities and reachability checks (p_exploit, segment edges, C2 reachability). **Defender events** are externally-injected transitions (isolate, segment, patch, sinkhole, restore).

13. Final Scenario Blueprint (implementation-ready)

A consolidated spec the development team can build against.

13.1 Core engine objects

- **Host** — {id, role, segment_id, state, persistent, recovery_disabled, encrypted_file_count, file_total, timestamps}
- **Segment/VLAN** — {id, host_ids[], allow_445_edges[]} (reachability graph)
- **WormAutomaton** — per-host loop implementing Section 7 (Attacker Journey) actions as state transitions + telemetry emissions

- **TelemetryBus** — emits typed events (`scan`, `smb_negotiation`, `exploit_signature`, `file_write`, `process_create`, `service_create`, `dns_callback`, `shadow_delete`, `mass_rename`, `ransom_note`, `service_down`) to the SOC view
- **SOCConsole** — consumes TelemetryBus, applies detection rules, surfaces alerts in the funnel of Section 5
- **DefenderController** — injects DP actions (Section 11) that fire defender-driven transitions
- **ImpactLedger** — runs the cost model of Section 9 in real time
- **Clock** — drives the timeline (Section 10) with adjustable time-scale + pause + branch

13.2 Tick loop (recommended)

```
on each tick:
    for host in infected_hosts:
        worm_automaton.step(host)          # may scan, exploit, deploy,
encrypt
    emit relevant telemetry
    soc_console.evaluate(telemetry_window)  # raise/escalate alerts
    defender_controller.apply_pending_actions() # may transition hosts
    impact_ledger.recompute()               # update files/hosts/$$/downtime
    state_recorder.snapshot()               # for timeline scrubbing + debrief
```

13.3 Required views (UI surfaces)

1. **Network Topology Map** — node-state colors, scan/strike/spread animations, segment containers, R-gauge, infection-count curve.
2. **SOC Console** — alert feed (funnel-ordered), per-host telemetry drill-down, correlation hints.
3. **Defender Action Panel** — the six decision points with cost/consequence preview.
4. **Victim Host View** — selectable host showing the per-host experience of Section 4 (files flipping to locked, ransom-note popup, lock screen).
5. **Business Impact Dashboard** — encrypted files, impacted hosts, service health, downtime clock, live financial counter.
6. **Timeline / Debrief** — scrubber over recorded snapshots; overlays learner actions vs. cost curve; outcome band (Contained/Degraded/Catastrophic).

13.4 Tunable parameters (instructor/config layer)

```
vulnerable_density, p_exploit, scan_speed, time_scale, files_per_host,
cost_per_user_hour, reimage_cost, critical_service_revenue_per_min,
kill_switch_enabled, segmentation_topology, start_host.
```

13.5 Win/teach conditions

- **Primary learning objective:** demonstrate that **early containment** (DP-1/DP-2, segmentation at DP-4, sinkhole at DP-3) collapses the cost curve, while late reaction only limits recovery cost.
- **Scoring:** weighted on (time-to-first-correct-action) × (blast radius prevented) × (critical services preserved).

- **Replay value:** randomize `start_host`, `vulnerable_density`, and segmentation topology so each run differs.

13.6 Explicit non-goals (safety boundaries baked into the build)

- No real network traffic, no real SMB calls, no scanning of real hosts — everything is in-engine simulation.
- No exploit code, shellcode, or cryptographic payload — exploitation is a probability gate that *emits a signature event*; encryption is a *file-state flip*, not a real cipher operation.
- The "kill-switch domain," "shadow-copy deletion," etc. are modeled as **named telemetry events and state flags**, not as executable commands.

Appendix A — Telemetry-to-detection mapping (quick reference)

Telemetry event	Maps to stage	Detection rule class
scan (many 445 conns, 1 source)	Discovery	Horizontal port-scan
smb_negotiation (SMBv1 dialect)	Target ID	Legacy-protocol use
exploit_signature	Exploit	IDS/IPS signature
file_write (system dir) + process_create	Payload	EDR file/process anomaly
service_create / autorun change	Persistence	New-persistence alert
dns_callback (rare domain)	C2	DNS/NRD anomaly
scan from many sources	Lateral movement	Multi-source correlation
shadow_delete	Disable recovery	Ransomware-precursor rule
mass_rename + ransom_note	Encryption	Mass-modify / note-signature
service_down	Impact	Availability monitor

Appendix B — MITRE ATT&CK alignment (reference only)

T1046 (Discovery), T1210 (Exploit of remote service), T1059/003 (command execution / payload + persistence), T1071/001 (C2 over web protocol), T1021/002 (lateral movement via SMB shares), T1490 (inhibit recovery), T1486 (data encrypted for impact). Used here purely to anchor stage semantics — the document itself is a simulation design spec, not an ATT&CK report.